towardsdatascience.com

How to Make Your LLM More Accurate with RAG & Fine-Tuning

Sarah Schürch

14–18 minutes

Imagine studying a module at university for a semester. At the end, after an intensive learning phase, you take an exam – and you can recall the most important concepts without looking them up.

Now imagine the second situation: You are asked a question about a new topic. You don't know the answer straight away, so you pick up a book or browse a wiki to find the right information for the answer.

These two analogies represent two of the most important methods for improving the basic model of an LLM or adapting it to specific tasks and areas: **Retrieval Augmented Generation (RAG) and Fine-Tuning.**

But which example belongs to which method?

That's exactly what I'll explain in this article: After that, you'll know what RAG and fine-tuning are, the most important differences and which method is suitable for which application.

Let's dive in!

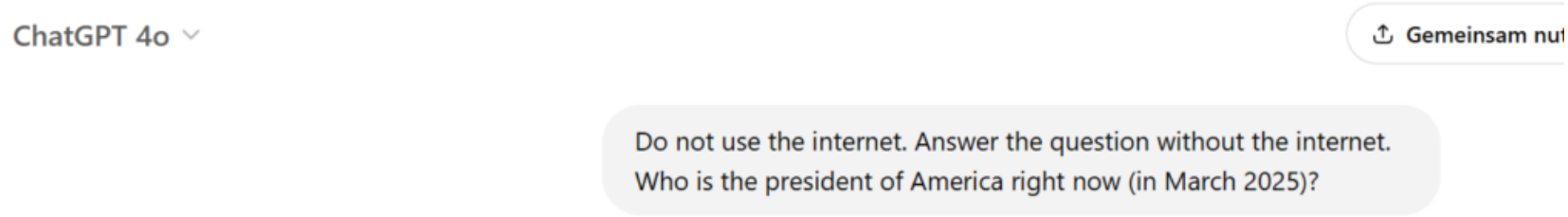
Table of contents

- 1. Basics: What is RAG? What is fine-tuning?
- 2. Differences between RAG and fine-tuning
- 3. Ways to build a RAG model
- 4. Options for fine-tuning a model
- 5. When is RAG recommended? When is fine-tuning recommended?
- Final Thoughts
- Where can you continue learning?

1. Basics: What is RAG? What is fine-tuning?

Large Language Models (LLMs) such as ChatGPT from OpenAI, Gemini from Google, Claude from Anthropics or Deepseek are incredibly powerful and have established themselves in everyday work over an extremely short time.

One of their biggest limitations is that their knowledge is limited to training. A model that was trained in 2024 does not know events from 2025. If we ask the 4o model from ChatGPT who the current US President is and give the clear instruction that the Internet should not be used, we see that it cannot answer this question with certainty:



Without using the internet, I cannot provide a definitive answer. However, based on past election cycles, the president of the United States in March 2025 would be the winner of the 2024 presidential election, who was inaugurated on January 20, 2025. The major candidates in the 2024 election included Joe Biden (the incumbent president at the time) and Donald Trump (a former president).

To know for sure who the current president is in March 2025, you would need to check an official or recent news source.

Screenshot taken by the author

In addition, the models cannot easily access company-specific information, such as internal guidelines or current technical documentation.

This is exactly where RAG and fine-tuning come into play.

Both methods make it possible to adapt an LLM to specific requirements:

RAG — The model remains the same, the input is improved

An LLM with Retrieval Augmented Generation (RAG) remains unchanged.

However, it gains access to an external knowledge source and can therefore retrieve information that is not stored in its model parameters. RAG extends the model in the inference phase by using external data sources to provide the latest or specific information. The inference phase is the moment when the model generates an answer.

This allows the model to stay up to date without retraining.

How does it work?

1. A user question is asked.
2. The query is converted into a vector representation.
3. A retriever searches for relevant text sections or data records in an external data source. The documents or FAQS are often stored in a vector database.
4. The content found is transferred to the model as additional context.
5. The LLM generates its answer on the basis of the retrieved and current information.

The key point is that the LLM itself remains unchanged and the internal weights of the LLM remain the same.

Let's assume a company uses an internal AI-powered support chatbot.

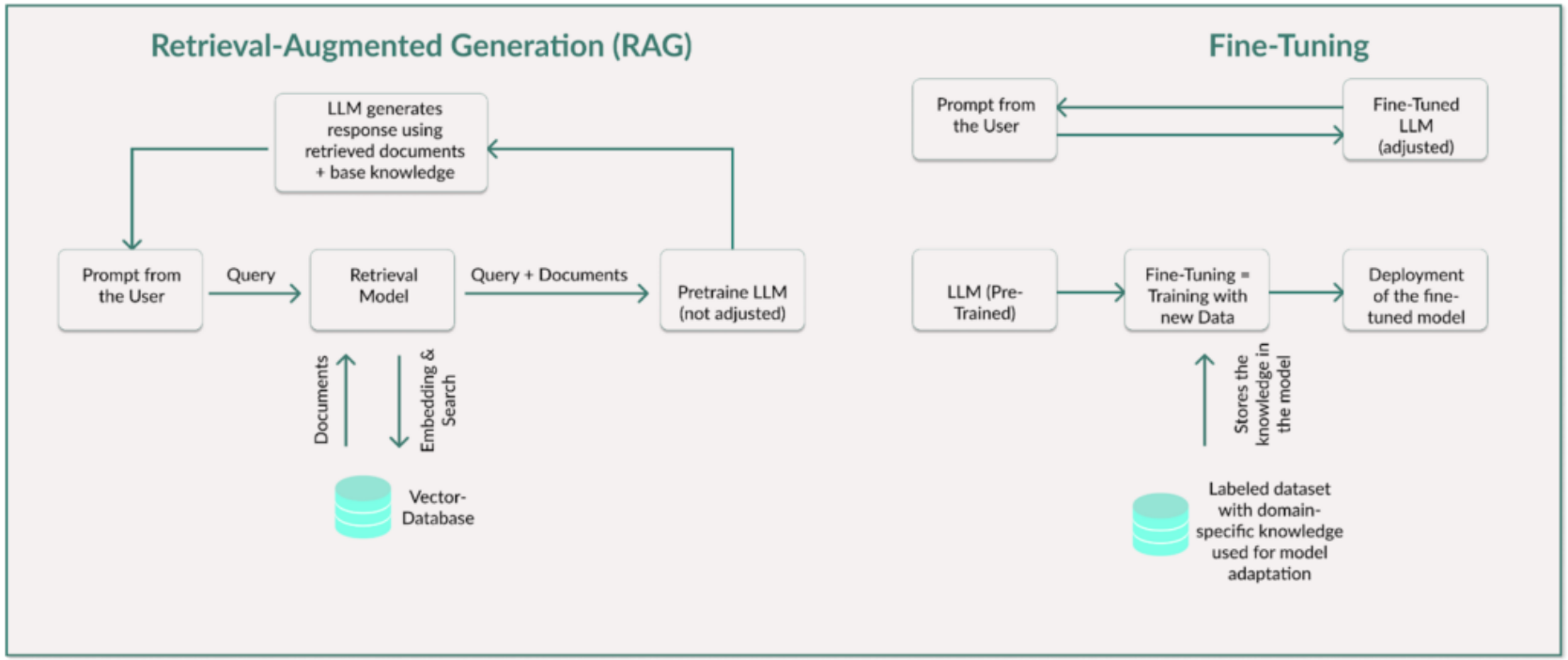
The chatbot helps employees to answer questions about company policies, IT processes or HR topics. If you would ask ChatGPT a question about your company (e.g. How many vacation days do I have left?), the model would logically not give you back a meaningful answer. A classic LLM without RAG would know nothing about the company – it has never been trained with this data.

This changes with RAG: The chatbot can search an external database of current company policies for the most relevant documents (e.g. PDF files, wiki pages or internal FAQs) and provide specific answers.

RAG works similarly as when we humans look up specific information in a library or Google search – but in real-time.

A student who is asked about the meaning of CRUD quickly looks up the Wikipedia article and answers Create, Read, Update and Delete – just like a RAG model retrieves relevant documents. This process allows both humans and AI to provide informed answers without memorizing everything.

And this makes RAG a powerful tool for keeping responses accurate and current.



Own visualization by the author

Fine-tuning — The model is trained and stores knowledge permanently

Instead of looking up external information, an LLM can also be directly updated with new knowledge through fine-tuning.

Fine-tuning is used during the training phase to provide the model with additional domain-specific knowledge. An existing base model is further trained with specific new data. As a result, it “learns” specific content and internalizes technical terms, style or certain content, but retains its general understanding of language.

This makes fine-tuning an effective tool for customizing LLMs to specific needs, data or tasks.

How does this work?

1. The LLM is trained with a specialized data set. This data set contains specific knowledge about a domain or a task.
2. The model weights are adjusted so that the model stores the new knowledge directly in its parameters.
3. After training, the model can generate answers without the need for external sources.

Let's now assume we want to use an LLM that provides us with expert answers to legal questions.

To do this, this LLM is trained with legal texts so that it can provide precise answers after fine-tuning. For example, it learns complex terms such as "intentional tort" and can name the appropriate legal basis in the context of the relevant country. Instead of just giving a general definition, it can cite relevant laws and precedents.

This means that you no longer just have a general LLM like GPT-4o at your disposal, but a useful tool for legal decision-making.

If we look again at the analogy with humans, fine-tuning is comparable to having internalized knowledge after an intensive learning phase.

After this learning phase, a computer science student knows that the term CRUD stands for Create, Read, Update, Delete. He or she can explain the concept without needing to look it up. The general vocabulary has been expanded.

This internalization allows for faster, more confident responses—just like a fine-tuned LLM.

2. Differences between RAG and fine-tuning

Both methods improve the performance of an LLM for specific tasks.

Both methods require well-prepared data to work effectively.

And both methods help to reduce hallucinations – the generation of false or fabricated information.

But if we look at the table below, we can see the differences between these two methods:

RAG is particularly flexible because the model can always access up-to-date data without having to be retrained. It requires less computational effort in advance, but needs more resources while answering a question (inference). The latency can also be higher.

Fine-tuning, on the other hand, offers faster inference times because the knowledge is stored directly in the model weights and no external search is necessary. The major disadvantage is that training is time-consuming and expensive and requires large amounts of high-quality training data.

RAG provides the model with tools to look up knowledge when needed without changing the model itself, whereas fine-tuning stores the additional knowledge in the model with adjusted parameters and weights.

Key Differences RAG and Fine-Tuning

CRITERION	RETRIEVAL AUGMENTED GENERATION (RAG)	FINE-TUNING
When knowledge updates	Can access latest data at runtime. The model has not to be retrained for new info.	Requires retraining to include new data (the model has static knowledge after training).
Domain expertise	The model relies on the base model & the knowledge base. -->Good expertise, as long as the necessary knowledge is available in the documents.	The model internalizes domain & task nuances. --> High expertise

Data Requirements	Needs a curated knowledge base (documents, passages) and an indexing process, but not labeled training pairs	Needs domain-specific training examples and substantial data processing.
Up-front Compute Cost	No training required. But most compute is at inference time for retrieval. --> Low	Requires GPU-intensive training phase to adjust the model weights. --> High
Inference Efficiency	Needs per query embedding, search in knowledge base and handling context --> Can increase latency	Model just can be run. No external lookups needed --> Fast: Good for low-latency/ high-throughput
Handling of Private Data	Private data stays in secure datastore and is fetched as needed --> Easier to comply with data governance.	Training data is stored in model weights. --> Harder to ensure that certain private data is not reproduced by a model.
Transparency	The model can return the documents it used to generate an answer --> More transparent	The answers are not easy to attribute to a specific training example. --> More opaque: It's more difficult to explain why the model said X

Own visualization by the author

3. Ways to build a RAG model

A popular framework for building a Retrieval Augmented Generation (RAG) pipeline is LangChain. This framework facilitates the linking of LLM calls with a retrieval system and makes it possible to retrieve information from external sources in a targeted manner.

How does RAG work technically?

1. Query embedding

In the first step, the user request is converted into a vector using an embedding model. This is done, for example, with text-embedding-ada-002 from OpenAI or all-MiniLM-L6-v2 from Hugging Face.

This is necessary because vector databases do not search through conventional texts, but instead calculate semantic similarities between numerical representations (embeddings). By converting the user query into a vector, the system can not only search for exactly matching terms, but also recognize concepts that are similar in content.

2. Search in the vector database

The resulting query vector is then compared with a vector database. The aim is to find the most relevant information to answer the question.

This similarity search is carried out using Approximate Nearest Neighbors (ANN) algorithms. Well-known open source tools for this task are, for example, FAISS from Meta for high-performance similarity searches in large data sets or ChromaDB for small to medium-sized retrieval tasks.

3. Insertion into the LLM context

In the third step, the retrieved documents or text sections are integrated into the prompt so that the LLM generates its response based on this information.

4. Generation of the response

The LLM now combines the information received with its general language vocabulary and generates a context-specific response.

An alternative to LangChain is the Hugging Face Transformer Library, which provides specially

developed RAG classes:

- ‘RagTokenizer’ tokenizes the input and the retrieval result. The class processes the text entered by the user and the retrieved documents.
- The ‘RagRetriever’ class performs the semantic search and retrieval of relevant documents from the predefined knowledge base.
- The ‘RagSequenceForGeneration’ class takes the documents provided, integrates them into the context and transfers them to the actual language model for answer generation.

4. Options for fine-tuning a model

While an LLM with RAG uses external information for the query, with fine-tuning we change the model weights so that the model permanently stores the new knowledge.

How does fine-tuning work technically?

1. Preparation of the training data

Fine-tuning requires a high-quality collection of data. This collection consists of inputs and the desired model responses. For a chatbot, for example, these can be question-answer pairs. For medical models, this could be clinical reports or diagnostic data. For a legal AI, these could be legal texts and judgments.

Let’s take a look at an example: If we look at the documentation of OpenAI, we see that these models use a standardized chat format with roles (system, user, assistant) during fine-tuning. The data format of these question-answer pairs is JSONL and looks like this, for example:

```
{"messages": [{"role": "system", "content": "Du bist ein medizinischer
```

```
Assistent."}, {"role": "user", "content": "Was sind Symptome einer
Grippe?"}, {"role": "assistant", "content": "Die häufigsten Symptome
einer Grippe sind Fieber, Husten, Muskel- und Gelenkschmerzen."}]}
```

Other models use other data formats such as CSV, JSON or PyTorch datasets.

2. Selection of the base model

We can use a pre-trained LLM as a starting point. These can be closed-source models such as GPT-3.5 or GPT-4 via OpenAI API or open-source models such as DeepSeek, LLaMA, Mistral or Falcon or T5 or FLAN-T5 for NLP tasks.

3. Training of the model

Fine-tuning requires a lot of computing power, as the model is trained with new data to update its weights. Especially large models such as GPT-4 or LLaMA 65B require powerful GPUs or TPUs.

To reduce the computational effort, there are optimized methods such as LoRA (Low-Rank Adaption), where only a small number of additional parameters are trained, or QLoRA (Quantized LoRA), where quantized model weights (e.g. 4-bit) are used.

4. Model deployment & use

Once the model has been trained, we can deploy it locally or on a cloud platform such as Hugging Face Model Hub, AWS or Azure.

5. When is RAG recommended? When is fine-tuning recommended?

RAG and fine-tuning have different advantages and disadvantages and are therefore suitable for different use cases:

RAG is particularly suitable when content is updated dynamically or frequently.

For example, in FAQ chatbots where information needs to be retrieved from a knowledge database that is constantly expanding. Technical documentation that is regularly updated can also be efficiently integrated using RAG – without the model having to be constantly retrained.

Another point is resources: If limited computing power or a smaller budget is available, RAG makes more sense as no complex training processes are required.

Fine-tuning, on the other hand, is suitable when a model needs to be tailored to a specific company or industry.

The response quality and style can be improved through targeted training. For example, the LLM can then generate medical reports with precise terminology.

The basic rule is: RAG is used when the knowledge is too extensive or too dynamic to be fully integrated into the model, while fine-tuning is the better choice when consistent, task-specific behavior is required.

And then there's RAFT — the magic of combination

What if we combine the two?

That's exactly what happens with Retrieval Augmented Fine-Tuning (RAFT).

The model is first enriched with domain-specific knowledge through fine-tuning so that it understands the correct terminology and structure. The model is then extended with RAG so that it can integrate specific and up-to-date information from external data sources. This combination ensures both deep expertise and real-time adaptability.

Companies use the advantages of both methods.

Final thoughts

Both methods—RAG and fine-tuning—extend the capabilities of a basic LLM in different ways.

Fine-tuning specializes the model for a specific domain, while RAG equips it with external knowledge. The two methods are not mutually exclusive and can be combined in hybrid approaches. Looking at computational costs, fine-tuning is resource-intensive upfront but efficient during operation, whereas RAG requires fewer initial resources but consumes more during use.

RAG is ideal when knowledge is too vast or dynamic to be integrated directly into the model. Fine-tuning is the better choice when stability and consistent optimization for a specific task are required. Both approaches serve distinct but complementary purposes, making them valuable tools in AI applications.

On my Substack, I regularly write summaries about the published articles in the fields of Tech, Python, Data Science, Machine Learning and AI. If you're interested, take a look or subscribe.

Where can you continue learning?

- OpenAI Documentation – Fine-tuning
- Hugging Face Blog QLoRA
- Microsoft Learn – Augment LLMs with RAG or fine-tuning
- IBM Technology YouTube – RAG vs. Fine Tuning
- DataCamp Blog – What is RAFT?
- DataCamp Blog – RAG vs. Fine-Tuning